MIT ACADEMY OF ENGINEERING

DEPARTMENT OF CSE AIML

Assignment:4

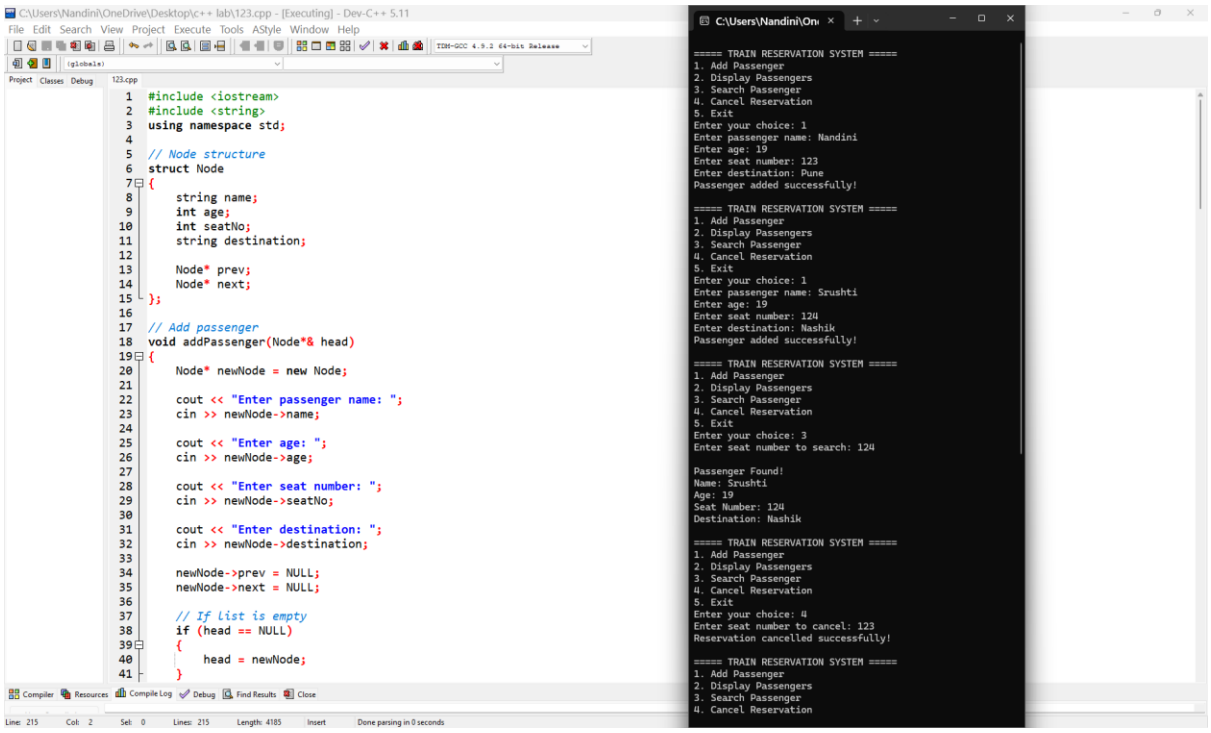
Name : Nandini Ravindra Chimandare

Div/Batch : AIML B1

PRN : 202501110108

- **Train Reservation System (Doubly Linked List)**

Implementation:



```cpp
#include <iostream>
#include <string>
using namespace std;

// Node structure
struct Node
{
    string name;
    int age;
    int seatNo;
    string destination;

    Node* prev;
    Node* next;
};

// Add passenger
void addPassenger(Node*& head)
{
    Node* newNode = new Node;

    cout << "Enter passenger name: ";
    cin >> newNode->name;

    cout << "Enter age: ";
    cin >> newNode->age;

    cout << "Enter seat number: ";
    cin >> newNode->seatNo;

    cout << "Enter destination: ";
    cin >> newNode->destination;

    newNode->prev = NULL;
    newNode->next = NULL;

    // If list is empty
    if (head == NULL)
    {
        head = newNode;
    }
```

```
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 1
Enter passenger name: Nandini
Enter age: 19
Enter seat number: 123
Enter destination: Pune
Passenger added successfully!

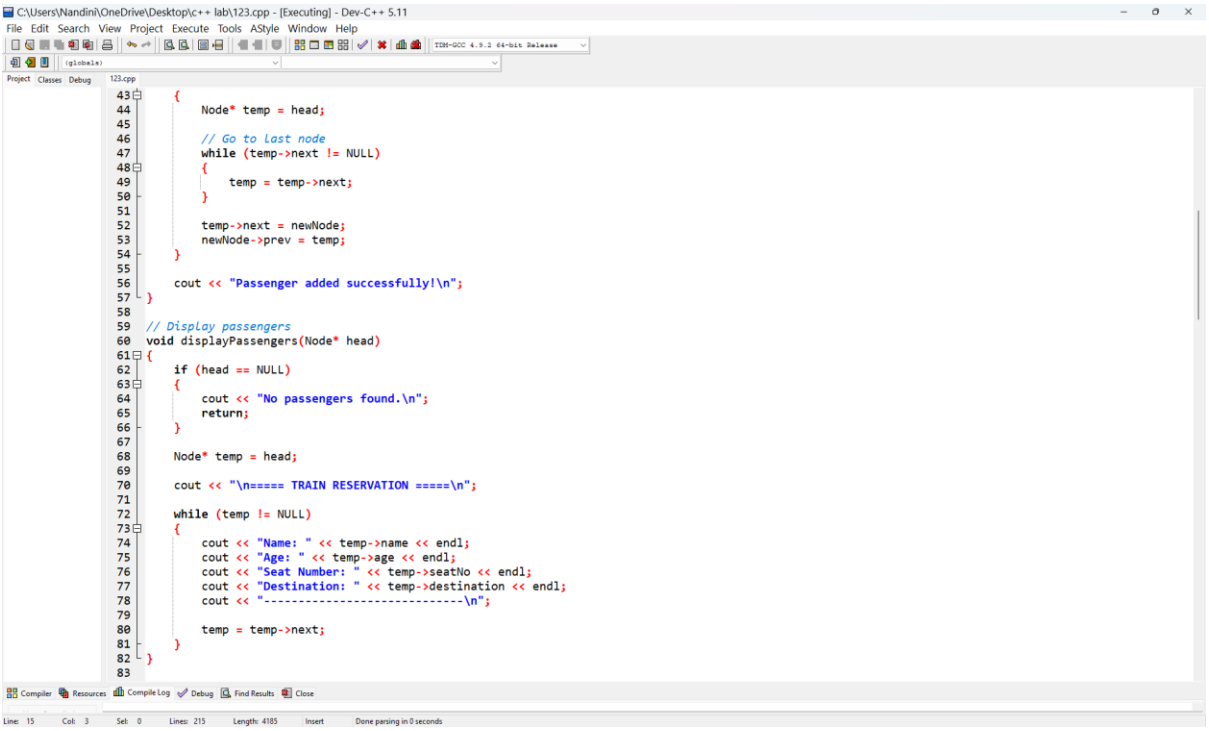
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 1
Enter passenger name: Srushti
Enter age: 19
Enter seat number: 124
Enter destination: Nashik
Passenger added successfully!

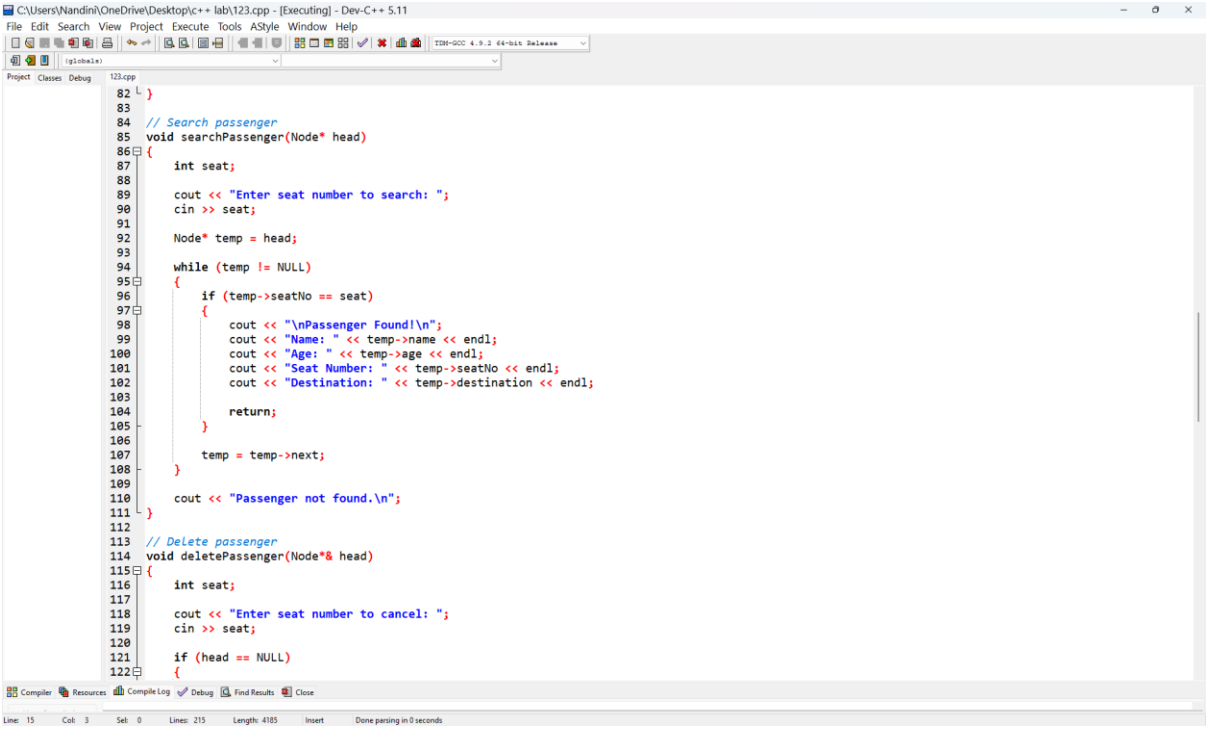
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 3
Enter seat number to search: 124

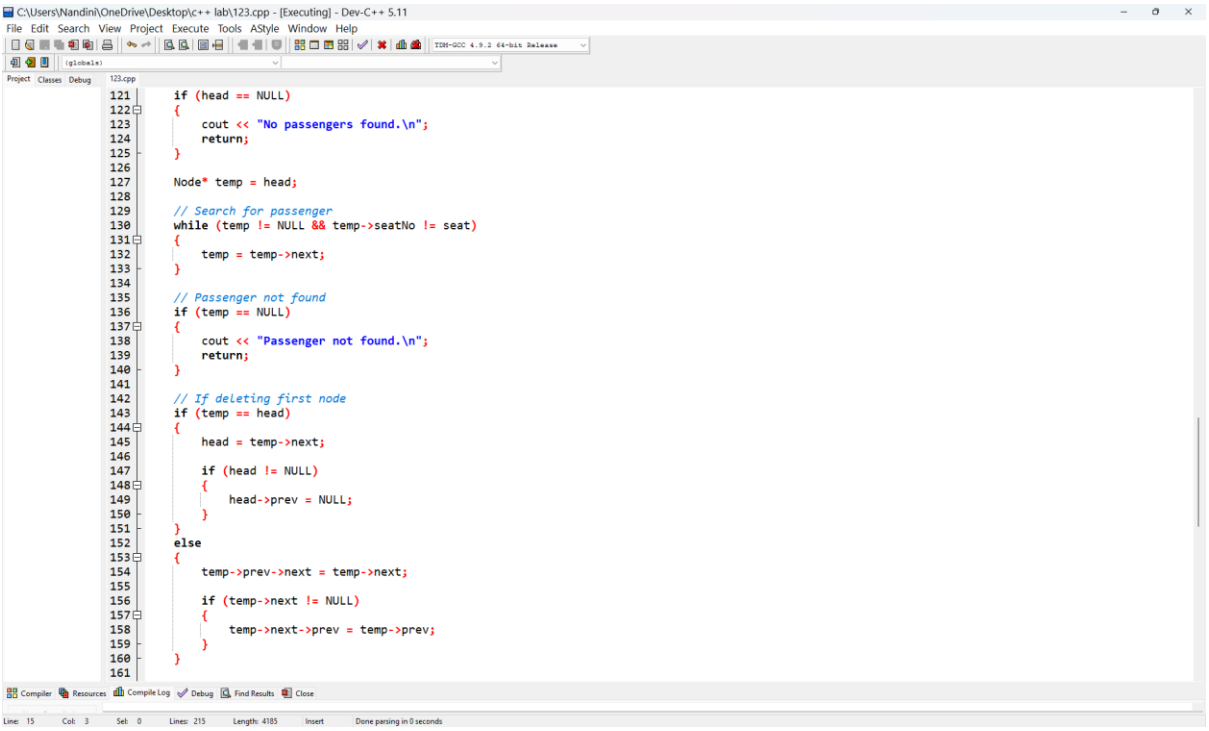
Passenger Found!
Name: Srushti
Age: 19
Seat Number: 124
Destination: Nashik

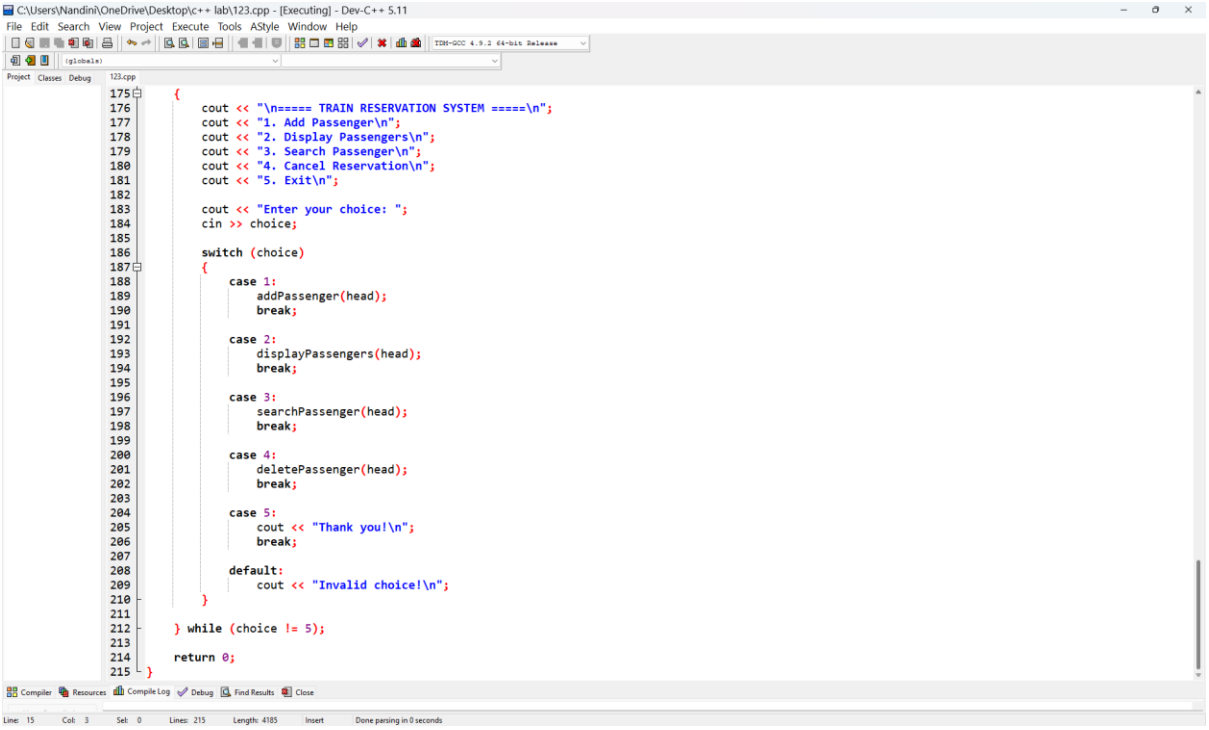
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 4
Enter seat number to cancel: 123
Reservation cancelled successfully!

===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
```

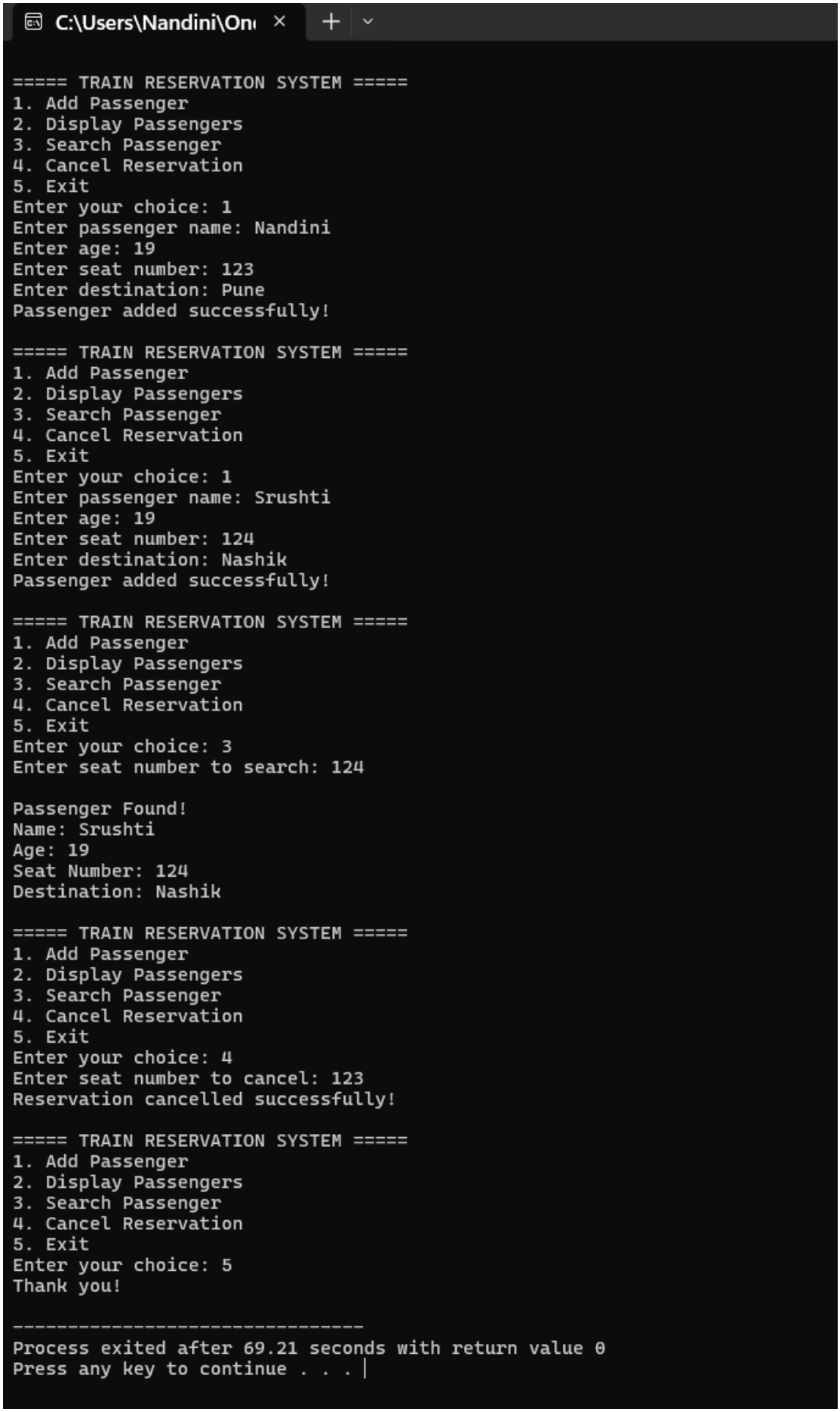

C:\Users\Nandini\OneDrive\Desktop\c++ lab\123.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
123.cpp
{
Node* temp = head;
// Go to last node
while (temp->next != NULL)
{
temp = temp->next;
}
temp->next = newNode;
newNode->prev = temp;
}
cout << "Passenger added successfully!\n";
}
// Display passengers
void displayPassengers(Node* head)
{
if (head == NULL)
{
cout << "No passengers found.\n";
return;
}
Node* temp = head;
cout << "\n===== TRAIN RESERVATION =====\n";
while (temp != NULL)
{
cout << "Name: " << temp->name << endl;
cout << "Age: " << temp->age << endl;
cout << "Seat Number: " << temp->seatNo << endl;
cout << "Destination: " << temp->destination << endl;
cout << "----------------------------\n";
temp = temp->next;
}
}
Compiler Resources Compile Log Debug Find Results Close
Done parsing in 0 seconds


C:\Users\Nandini\OneDrive\Desktop\c++ lab\123.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
123.cpp
}
// Search passenger
void searchPassenger(Node* head)
{
int seat;
cout << "Enter seat number to search: ";
cin >> seat;
Node* temp = head;
while (temp != NULL)
{
if (temp->seatNo == seat)
{
cout << "\nPassenger Found!\n";
cout << "Name: " << temp->name << endl;
cout << "Age: " << temp->age << endl;
cout << "Seat Number: " << temp->seatNo << endl;
cout << "Destination: " << temp->destination << endl;
return;
}
temp = temp->next;
}
cout << "Passenger not found.\n";
}
// Delete passenger
void deletePassenger(Node*& head)
{
int seat;
cout << "Enter seat number to cancel: ";
cin >> seat;
if (head == NULL)
{
Compiler Resources Compile Log Debug Find Results Close
Done parsing in 0 seconds


C:\Users\Nandini\OneDrive\Desktop\c++ lab\123.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
123.cpp
if (head == NULL)
{
cout << "No passengers found.\n";
return;
}
Node* temp = head;
// Search for passenger
while (temp != NULL && temp->seatNo != seat)
{
temp = temp->next;
}
// Passenger not found
if (temp == NULL)
{
cout << "Passenger not found.\n";
return;
}
// If deleting first node
if (temp == head)
{
head = temp->next;
if (head != NULL)
{
head->prev = NULL;
}
}
else
{
temp->prev->next = temp->next;
if (temp->next != NULL)
{
temp->next->prev = temp->prev;
}
}
Compiler Resources Compile Log Debug Find Results Close
Done parsing in 0 seconds


C:\Users\Nandini\OneDrive\Desktop\c++ lab\123.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
123.cpp
{
cout << "\n===== TRAIN RESERVATION SYSTEM =====\n";
cout << "1. Add Passenger\n";
cout << "2. Display Passengers\n";
cout << "3. Search Passenger\n";
cout << "4. Cancel Reservation\n";
cout << "5. Exit\n";
cout << "Enter your choice: ";
cin >> choice;
switch (choice)
{
case 1:
addPassenger(head);
break;
case 2:
displayPassengers(head);
break;
case 3:
searchPassenger(head);
break;
case 4:
deletePassenger(head);
break;
case 5:
cout << "Thank you!\n";
break;
default:
cout << "Invalid choice!\n";
}
} while (choice != 5);
return 0;
}
Compiler Resources Compile Log Debug Find Results Close
Done parsing in 0 seconds

Output:


C:\Users\Nandini\On
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 1
Enter passenger name: Nandini
Enter age: 19
Enter seat number: 123
Enter destination: Pune
Passenger added successfully!
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 1
Enter passenger name: Srushti
Enter age: 19
Enter seat number: 124
Enter destination: Nashik
Passenger added successfully!
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 3
Enter seat number to search: 124
Passenger Found!
Name: Srushti
Age: 19
Seat Number: 124
Destination: Nashik
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 4
Enter seat number to cancel: 123
Reservation cancelled successfully!
===== TRAIN RESERVATION SYSTEM =====
1. Add Passenger
2. Display Passengers
3. Search Passenger
4. Cancel Reservation
5. Exit
Enter your choice: 5
Thank you!

Process exited after 69.21 seconds with return value 0
Press any key to continue . . .

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Node structure
struct Node
{
   string name;
   int age;
   int seatNo;
   string destination;

   Node* prev;
   Node* next;
};

// Add passenger
void addPassenger(Node*& head)
{
   Node* newNode = new Node;

   cout << "Enter passenger name: ";
   cin >> newNode->name;

   cout << "Enter age: ";
   cin >> newNode->age;

   cout << "Enter seat number: ";
   cin >> newNode->seatNo;
```

```
    cout << "Enter destination: ";
    cin >> newNode->destination;

    newNode->prev = NULL;
    newNode->next = NULL;

    // If list is empty
    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node* temp = head;

        // Go to last node
        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
        newNode->prev = temp;
    }

    cout << "Passenger added successfully!\n";
}

// Display passengers
void displayPassengers(Node* head)
{
```

```
    if (head == NULL)
    {
        cout << "No passengers found.\n";
        return;
    }

    Node* temp = head;

    cout << "\n===== TRAIN RESERVATION =====\n";

    while (temp != NULL)
    {
        cout << "Name: " << temp->name << endl;
        cout << "Age: " << temp->age << endl;
        cout << "Seat Number: " << temp->seatNo << endl;
        cout << "Destination: " << temp->destination << endl;
        cout << "-----------------------------\n";

        temp = temp->next;
    }
}

// Search passenger
void searchPassenger(Node* head)
{
    int seat;

    cout << "Enter seat number to search: ";
    cin >> seat;

    Node* temp = head;
```

```
    while (temp != NULL)
    {
        if (temp->seatNo == seat)
        {
            cout << "\nPassenger Found!\n";
            cout << "Name: " << temp->name << endl;
            cout << "Age: " << temp->age << endl;
            cout << "Seat Number: " << temp->seatNo << endl;
            cout << "Destination: " << temp->destination << endl;

            return;
        }

        temp = temp->next;
    }

    cout << "Passenger not found.\n";
}

// Delete passenger
void deletePassenger(Node*& head)
{
    int seat;

    cout << "Enter seat number to cancel: ";
    cin >> seat;

    if (head == NULL)
    {
        cout << "No passengers found.\n";
```

```
    return;
}

Node* temp = head;

// Search for passenger
while (temp != NULL && temp->seatNo != seat)
{
    temp = temp->next;
}

// Passenger not found
if (temp == NULL)
{
    cout << "Passenger not found.\n";
    return;
}

// If deleting first node
if (temp == head)
{
    head = temp->next;

    if (head != NULL)
    {
        head->prev = NULL;
    }
}
else
{
    temp->prev->next = temp->next;
```

```
        if (temp->next != NULL)
        {
            temp->next->prev = temp->prev;
        }
    }

    delete temp;

    cout << "Reservation cancelled successfully!\n";
}

// Main function
int main()
{
    Node* head = NULL;

    int choice;

    do
    {
        cout << "\n===== TRAIN RESERVATION SYSTEM =====\n";
        cout << "1. Add Passenger\n";
        cout << "2. Display Passengers\n";
        cout << "3. Search Passenger\n";
        cout << "4. Cancel Reservation\n";
        cout << "5. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;
```

```
        switch (choice)
        {
            case 1:
                addPassenger(head);
                break;

            case 2:
                displayPassengers(head);
                break;

            case 3:
                searchPassenger(head);
                break;

            case 4:
                deletePassenger(head);
                break;

            case 5:
                cout << "Thank you!\n";
                break;

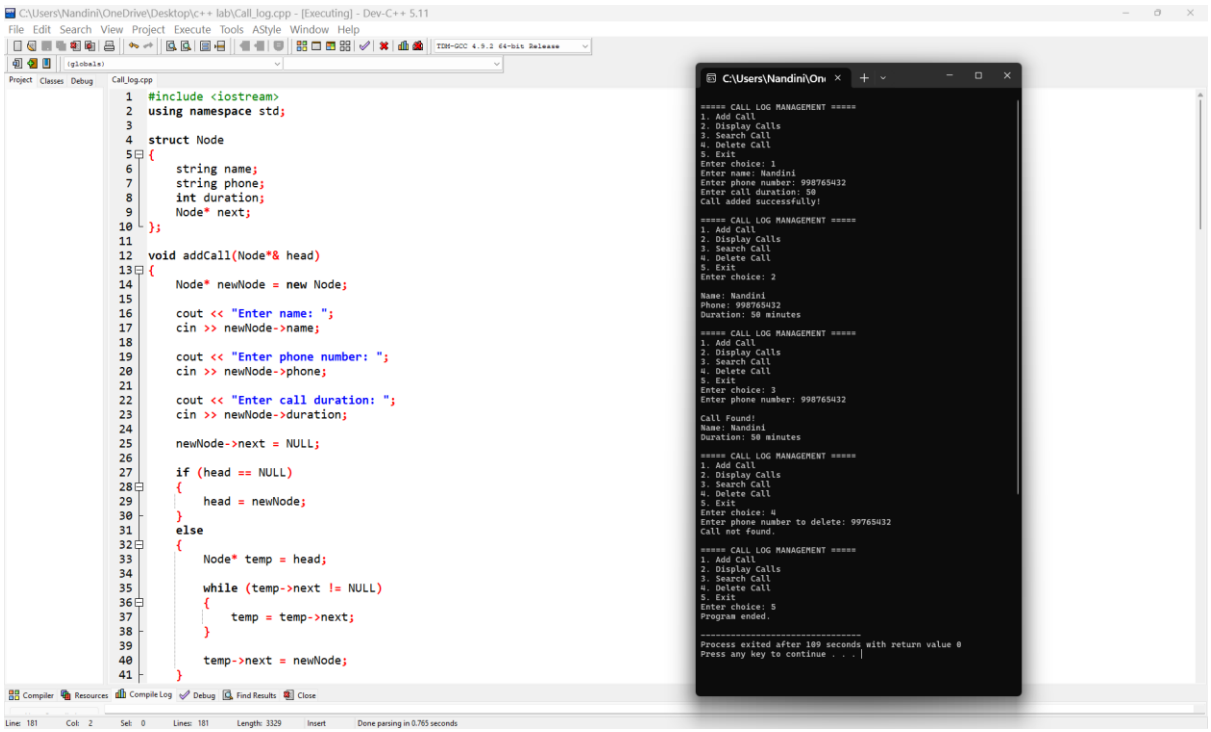
            default:
                cout << "Invalid choice!\n";
        }

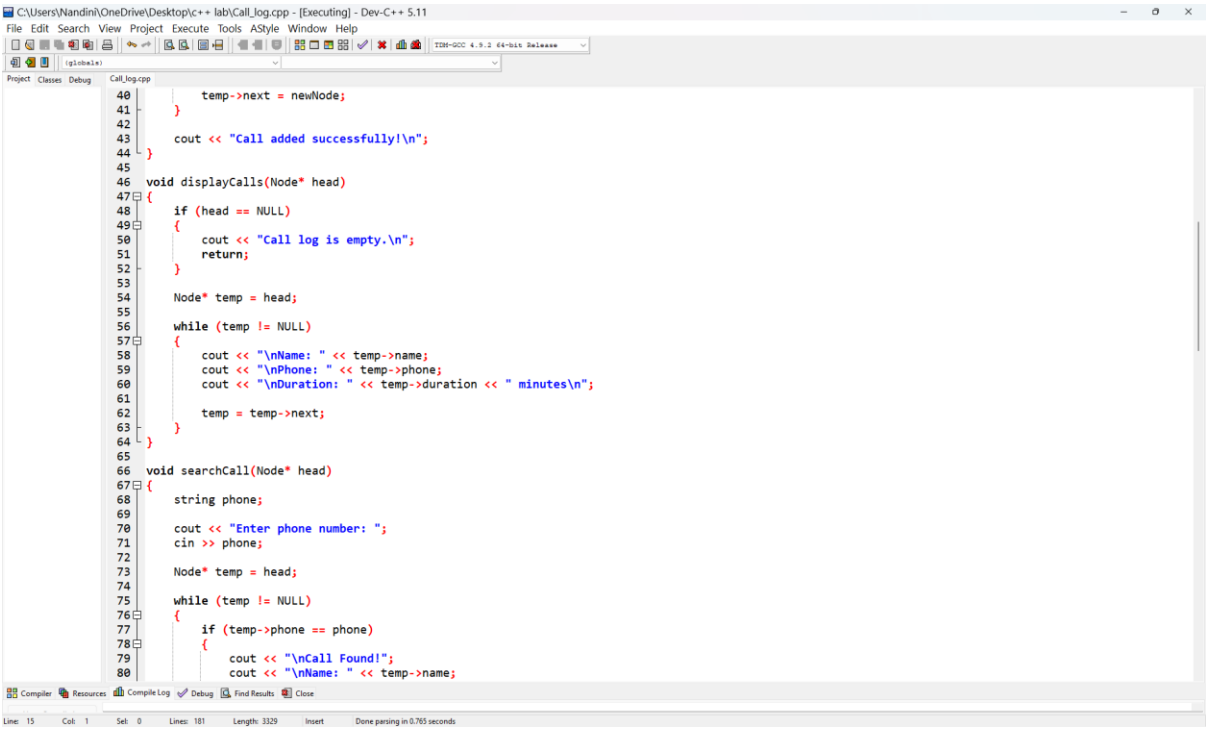
    } while (choice != 5);

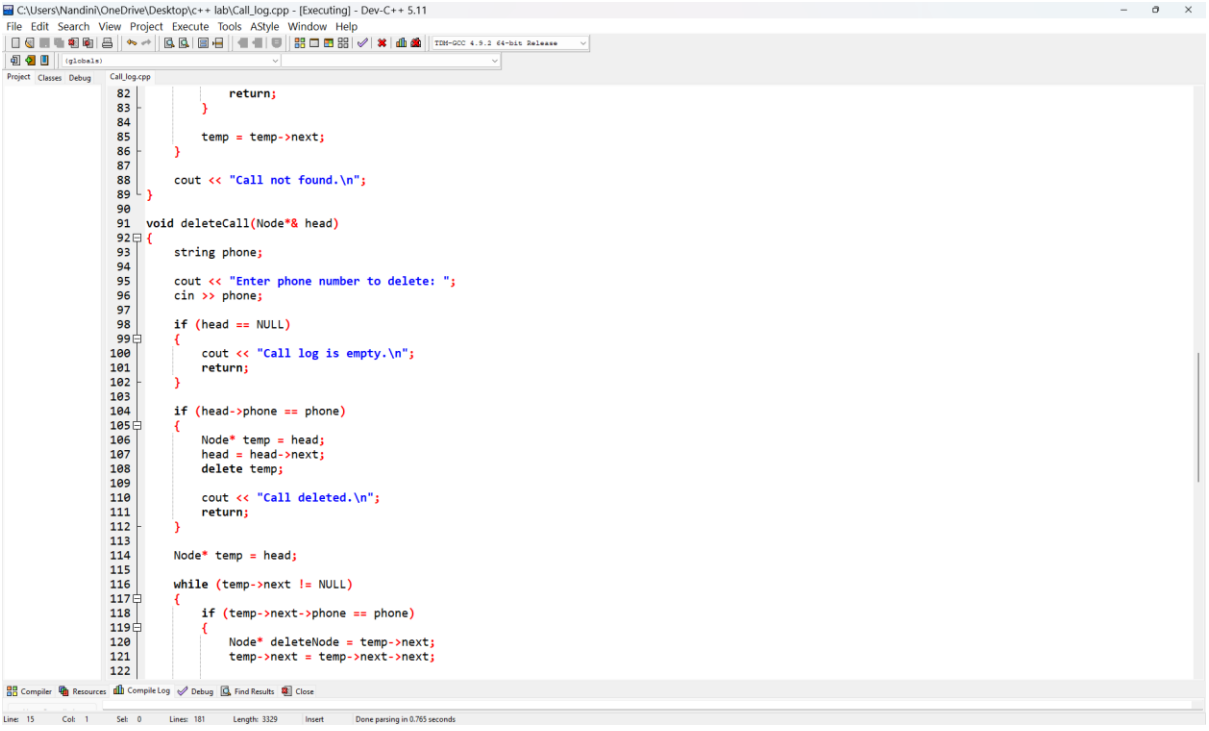
    return 0;
}
```

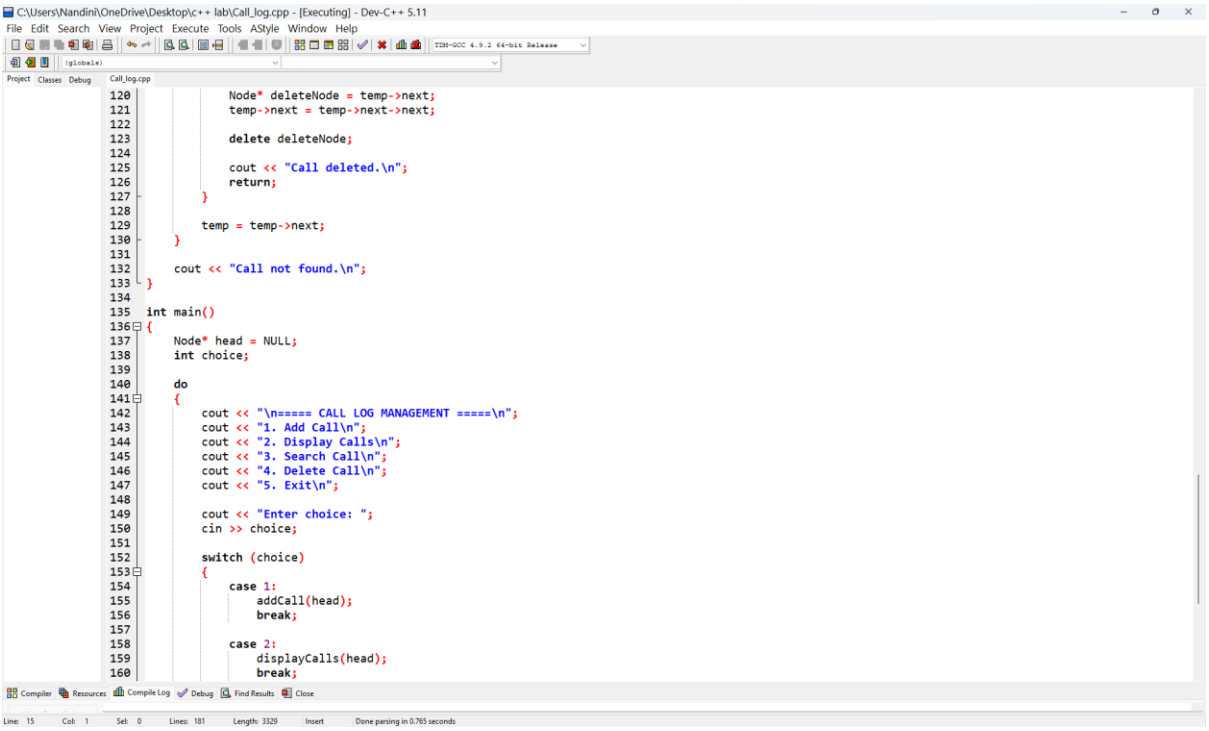
- **Call Log Management — Singly Linked List**

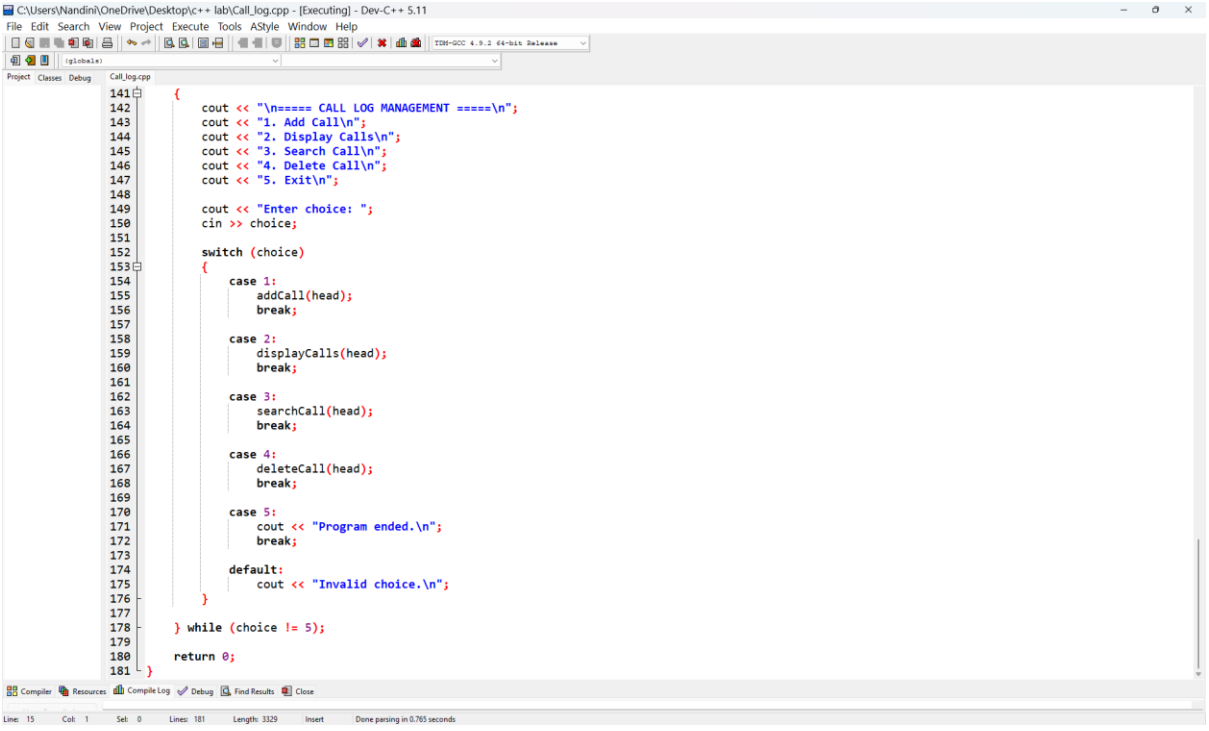
- **Implementation:**




C:\Users\Nandini\OneDrive\Desktop\c++ lab\Call_log.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
Project Classes Debug
Call_log.cpp
temp->next = newNode;
}
cout << "Call added successfully!\n";
}
void displayCalls(Node* head)
{
if (head == NULL)
{
cout << "Call log is empty.\n";
return;
}
Node* temp = head;
while (temp != NULL)
{
cout << "\nName: " << temp->name;
cout << "\nPhone: " << temp->phone;
cout << "\nDuration: " << temp->duration << " minutes\n";
temp = temp->next;
}
}
void searchCall(Node* head)
{
string phone;
cout << "Enter phone number: ";
cin >> phone;
Node* temp = head;
while (temp != NULL)
{
if (temp->phone == phone)
{
cout << "\nCall Found!";
cout << "\nName: " << temp->name;
Compiler Resources Compile Log Debug Find Results Close
Line: 15 Col: 1 Sel: 0 Lines: 181 Length: 3329 Insert Done parsing in 0.765 seconds


C:\Users\Nandini\OneDrive\Desktop\c++ lab\Call_log.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
Project Classes Debug
Call_log.cpp
return;
}
temp = temp->next;
}
cout << "Call not found.\n";
}
void deleteCall(Node*& head)
{
string phone;
cout << "Enter phone number to delete: ";
cin >> phone;
if (head == NULL)
{
cout << "Call log is empty.\n";
return;
}
if (head->phone == phone)
{
Node* temp = head;
head = head->next;
delete temp;
cout << "Call deleted.\n";
return;
}
Node* temp = head;
while (temp->next != NULL)
{
if (temp->next->phone == phone)
{
Node* deleteNode = temp->next;
temp->next = temp->next->next;
Compiler Resources Compile Log Debug Find Results Close
Line: 15 Col: 1 Sel: 0 Lines: 181 Length: 3329 Insert Done parsing in 0.765 seconds


C:\Users\Nandini\OneDrive\Desktop\c++ lab\Call_log.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
Call_log.cpp
Node* deleteNode = temp->next;
temp->next = temp->next->next;
delete deleteNode;
cout << "Call deleted.\n";
return;
temp = temp->next;
cout << "Call not found.\n";
int main()
Node* head = NULL;
int choice;
do
cout << "\n===== CALL LOG MANAGEMENT =====\n";
cout << "1. Add Call\n";
cout << "2. Display Calls\n";
cout << "3. Search Call\n";
cout << "4. Delete Call\n";
cout << "5. Exit\n";
cout << "Enter choice: ";
cin >> choice;
switch (choice)
case 1:
addCall(head);
break;
case 2:
displayCalls(head);
break;


C:\Users\Nandini\OneDrive\Desktop\c++ lab\Call_log.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
Call_log.cpp
cout << "\n===== CALL LOG MANAGEMENT =====\n";
cout << "1. Add Call\n";
cout << "2. Display Calls\n";
cout << "3. Search Call\n";
cout << "4. Delete Call\n";
cout << "5. Exit\n";
cout << "Enter choice: ";
cin >> choice;
switch (choice)
case 1:
addCall(head);
break;
case 2:
displayCalls(head);
break;
case 3:
searchCall(head);
break;
case 4:
deleteCall(head);
break;
case 5:
cout << "Program ended.\n";
break;
default:
cout << "Invalid choice.\n";
} while (choice != 5);
return 0;

- **Code**

```
#include <iostream>
using namespace std;

struct Node
{
   string name;
   string phone;
   int duration;
   Node* next;
};

void addCall(Node*& head)
{
   Node* newNode = new Node;

   cout << "Enter name: ";
   cin >> newNode->name;

   cout << "Enter phone number: ";
   cin >> newNode->phone;

   cout << "Enter call duration: ";
   cin >> newNode->duration;

   newNode->next = NULL;

   if (head == NULL)
   {
      head = newNode;
   }
```

```
    else
    {
        Node* temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
    }

    cout << "Call added successfully!\n";
}

void displayCalls(Node* head)
{
    if (head == NULL)
    {
        cout << "Call log is empty.\n";
        return;
    }

    Node* temp = head;

    while (temp != NULL)
    {
        cout << "\nName: " << temp->name;
        cout << "\nPhone: " << temp->phone;
        cout << "\nDuration: " << temp->duration << " minutes\n";
```

```
        temp = temp->next;
    }
}

void searchCall(Node* head)
{
    string phone;

    cout << "Enter phone number: ";
    cin >> phone;

    Node* temp = head;

    while (temp != NULL)
    {
        if (temp->phone == phone)
        {
            cout << "\nCall Found!";
            cout << "\nName: " << temp->name;
            cout << "\nDuration: " << temp->duration << " minutes\n";
            return;
        }

        temp = temp->next;
    }

    cout << "Call not found.\n";
}

void deleteCall(Node*& head)
{
```

```
string phone;

cout << "Enter phone number to delete: ";
cin >> phone;

if (head == NULL)
{
   cout << "Call log is empty.\n";
   return;
}

if (head->phone == phone)
{
   Node* temp = head;
   head = head->next;
   delete temp;

   cout << "Call deleted.\n";
   return;
}

Node* temp = head;

while (temp->next != NULL)
{
   if (temp->next->phone == phone)
   {
      Node* deleteNode = temp->next;
      temp->next = temp->next->next;

      delete deleteNode;
```

```
            cout << "Call deleted.\n";
            return;
        }

        temp = temp->next;
    }

    cout << "Call not found.\n";
}

int main()
{
    Node* head = NULL;
    int choice;

    do
    {
        cout << "\n===== CALL LOG MANAGEMENT =====\n";
        cout << "1. Add Call\n";
        cout << "2. Display Calls\n";
        cout << "3. Search Call\n";
        cout << "4. Delete Call\n";
        cout << "5. Exit\n";

        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
```

```
            addCall(head);
            break;

        case 2:
            displayCalls(head);
            break;

        case 3:
            searchCall(head);
            break;

        case 4:
            deleteCall(head);
            break;

        case 5:
            cout << "Program ended.\n";
            break;

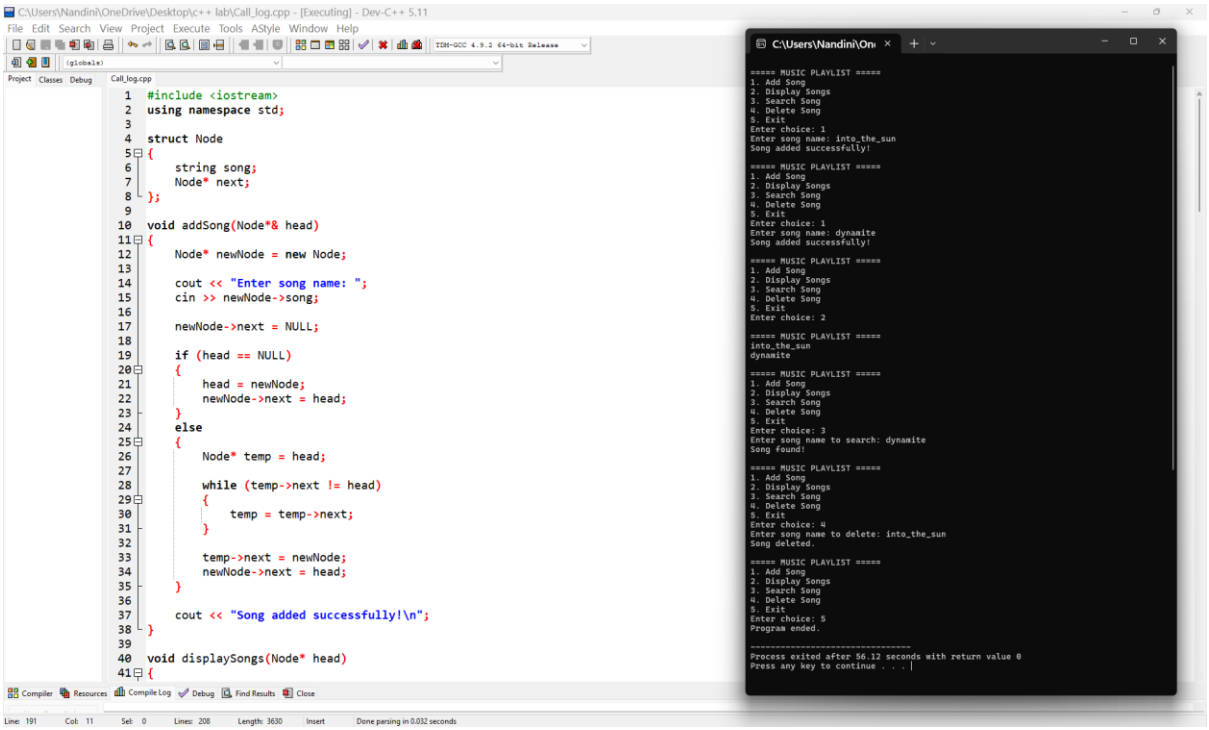
        default:
            cout << "Invalid choice.\n";
    }

  } while (choice != 5);

  return 0;
}
```

- **Music Playlist Management — Circular Linked List**

- **Implementation:**



```
#include <iostream>
using namespace std;

struct Node
{
    string song;
    Node* next;
};

void addSong(Node*& head)
{
    Node* newNode = new Node;

    cout << "Enter song name: ";
    cin >> newNode->song;

    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
    }
    else
    {
        Node* temp = head;

        while (temp->next != head)
        {
            temp = temp->next;
        }

        temp->next = newNode;
        newNode->next = head;
    }

    cout << "Song added successfully!\n";
}

void displaySongs(Node* head)
{
```

```
===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 1
Enter song name: into_the_sun
Song added successfully!

===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 1
Enter song name: dynamite
Song added successfully!

===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 2

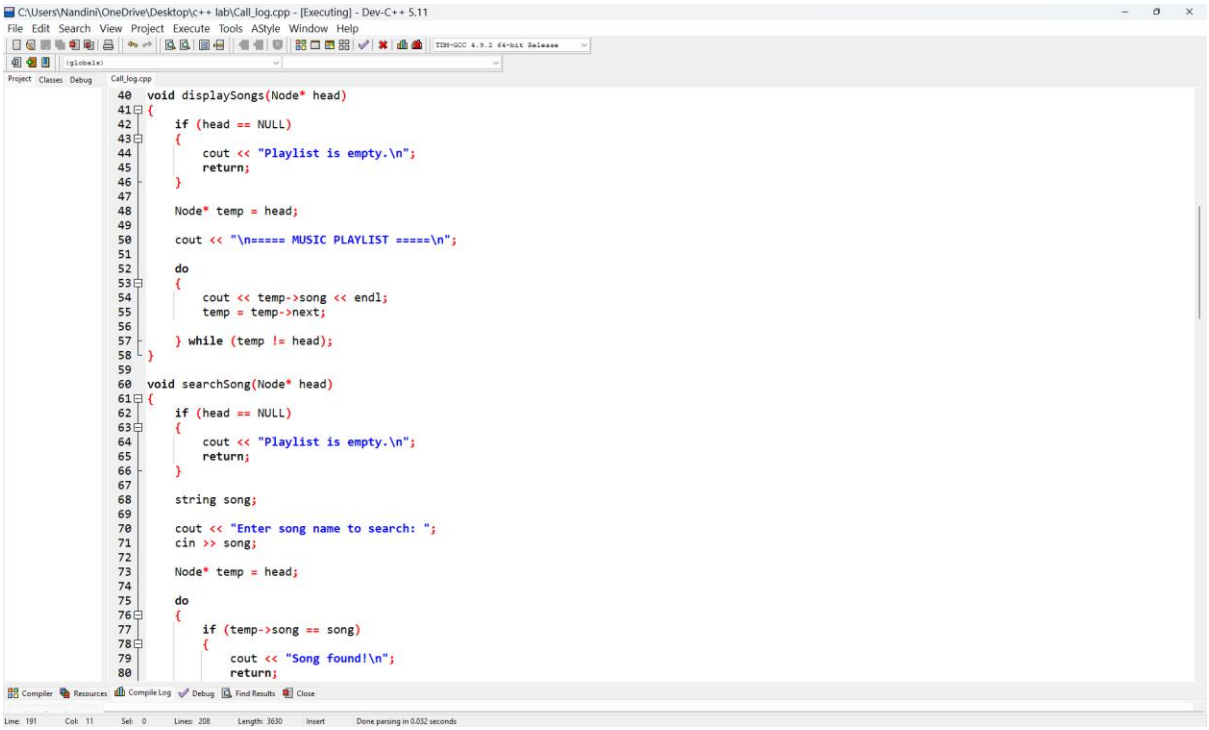
===== MUSIC PLAYLIST =====
into_the_sun
dynamite

===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 3
Enter song name to search: dynamite
Song found!

===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 4
Enter song name to delete: into_the_sun
Song deleted.

===== MUSIC PLAYLIST =====
1. Add Song
2. Display Songs
3. Search Song
4. Delete Song
5. Exit
Enter choice: 5
Program ended.

--------------------------------
Process exited after 56.12 seconds with return value 0
Press any key to continue . . .
```



```
void displaySongs(Node* head)
{
    if (head == NULL)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;

    cout << "\n===== MUSIC PLAYLIST =====\n";

    do
    {
        cout << temp->song << endl;
        temp = temp->next;

    } while (temp != head);
}

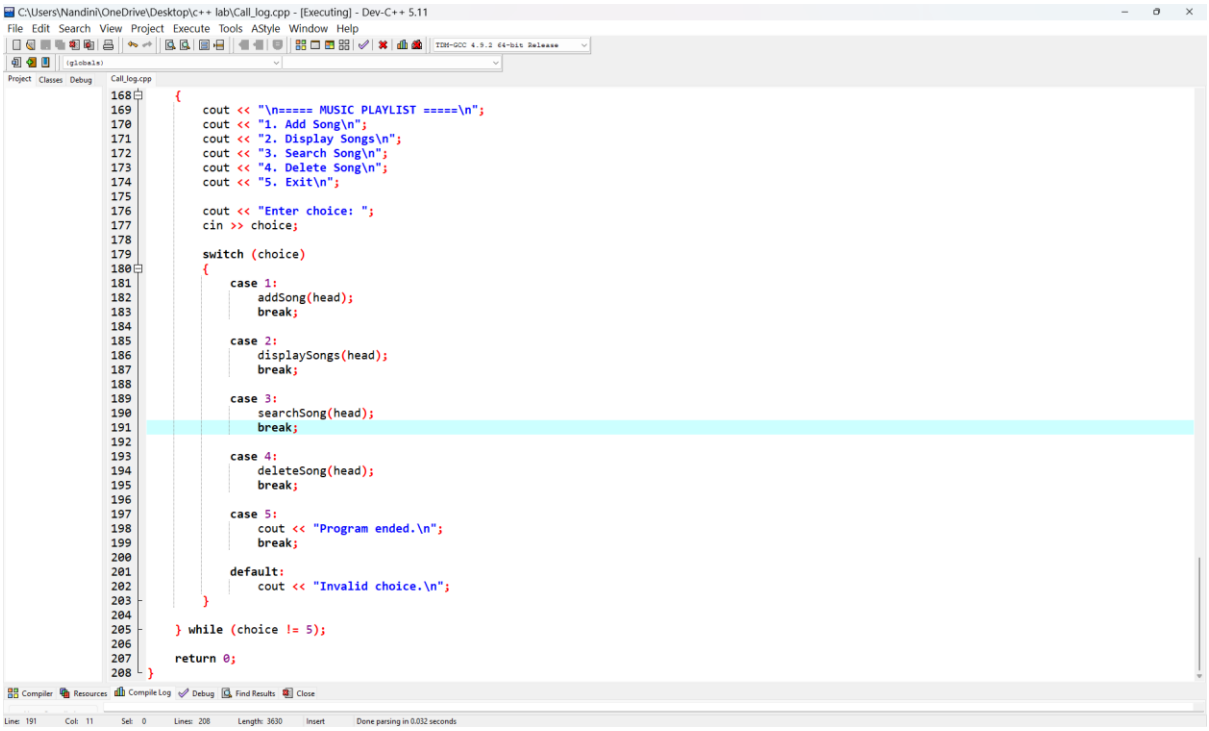
void searchSong(Node* head)
{
    if (head == NULL)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    string song;

    cout << "Enter song name to search: ";
    cin >> song;

    Node* temp = head;

    do
    {
        if (temp->song == song)
        {
            cout << "Song found!\n";
            return;
```



Code:

```
#include <iostream>
using namespace std;

struct Node
{
  string song;
  Node* next;
};

void addSong(Node*& head)
{
  Node* newNode = new Node;

  cout << "Enter song name: ";
  cin >> newNode->song;

  newNode->next = NULL;

  if (head == NULL)
  {
    head = newNode;
    newNode->next = head;
  }
  else
  {
```

```
        Node* temp = head;

        while (temp->next != head)
        {
            temp = temp->next;
        }

        temp->next = newNode;
        newNode->next = head;
    }

    cout << "Song added successfully!\n";
}

void displaySongs(Node* head)
{
    if (head == NULL)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    Node* temp = head;

    cout << "\n===== MUSIC PLAYLIST =====\n";

    do
    {
        cout << temp->song << endl;
        temp = temp->next;

    } while (temp != head);
}

void searchSong(Node* head)
{
    if (head == NULL)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    string song;

    cout << "Enter song name to search: ";
    cin >> song;

    Node* temp = head;
```

```
    do
    {
        if (temp->song == song)
        {
            cout << "Song found!\n";
            return;
        }

        temp = temp->next;

    } while (temp != head);

    cout << "Song not found.\n";
}

void deleteSong(Node*& head)
{
    if (head == NULL)
    {
        cout << "Playlist is empty.\n";
        return;
    }

    string song;

    cout << "Enter song name to delete: ";
    cin >> song;

    Node* current = head;
    Node* previous = NULL;

    // If only one song
    if (head->next == head)
    {
        if (head->song == song)
        {
            delete head;
            head = NULL;

            cout << "Song deleted.\n";
            return;
        }
    }

    // Delete first song
    if (head->song == song)
    {
        Node* last = head;
```

```
        while (last->next != head)
        {
            last = last->next;
        }

        Node* temp = head;
        head = head->next;

        last->next = head;

        delete temp;

        cout << "Song deleted.\n";
        return;
    }

    // Delete other songs
    current = head;

    while (current->next != head)
    {
        if (current->next->song == song)
        {
            Node* temp = current->next;
            current->next = temp->next;

            delete temp;

            cout << "Song deleted.\n";
            return;
        }

        current = current->next;
    }

    cout << "Song not found.\n";
}

int main()
{
    Node* head = NULL;
    int choice;

    do
    {
        cout << "\n===== MUSIC PLAYLIST =====\n";
        cout << "1. Add Song\n";
        cout << "2. Display Songs\n";
        cout << "3. Search Song\n";
```

```
        cout << "4. Delete Song\n";
        cout << "5. Exit\n";

        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                addSong(head);
                break;

            case 2:
                displaySongs(head);
                break;

            case 3:
                searchSong(head);
                break;

            case 4:
                deleteSong(head);
                break;

            case 5:
                cout << "Program ended.\n";
                break;

            default:
                cout << "Invalid choice.\n";
        }

    } while (choice != 5);

    return 0;
}
```